HR MANAGEMENT SYSTEM

Capstone Project – Final Report

HR Management & Workforce Analytics

Project Title: HR Management System

Submitted By: Pranjali Bharat Jadhav

Date: 24 February 2026

Batch Name: MS Dynamics

Instructor: Parth Shukla

Table of Contents

---

1. Problem Definition and Objectives

1.1 Background and Problem Statement

Modern organizations face significant challenges in managing their human resources efficiently. Traditional HR processes — such as employee onboarding, leave management, payroll tracking, and workforce analytics — are often handled through disconnected spreadsheets, paper-based forms, or outdated legacy systems. This leads to data inconsistencies, slow approvals, lack of visibility into workforce trends, and high administrative overhead.The HR Management System (HRMS) is developed to address these core pain points by providing a centralized, technology-driven platform that automates and streamlines all major HR operations. The system leverages a modern full-stack architecture built on C# ASP.NET Core Web API for the backend, SQL Server for data persistence, and a responsive Single Page Application (SPA) on the frontend. In a growing mid-sized organization, the Human Resource (HR) department is responsible for managing employee records, departmental structures, leave tracking, attendance management, and workforce reporting. As the organization expands in size and complexity, the volume of employee data increases significantly, making manual management processes inefficient and difficult to control. Traditionally, many HR departments rely on spreadsheets, email-based approvals, and disconnected record-keeping methods. While such systems may function at a small scale, they become unreliable and error-prone as the organization grows. The lack of a centralized, structured, and scalable HR platform leads to operational inefficiencies, data inconsistencies, and limited visibility into workforce trends.

The organization in this project was in the process of modernizing its HR operations to support better employee data management, structured leave workflows, and management-level reporting. There was a clear need for a modular and audit-ready system that could not only handle day-to-day HR activities but also provide insights to leadership for workforce planning and decision-making. Additionally, the system needed to be designed in a way that supports future integration with enterprise-level ERP systems and allows justified cloud integration for security and automation purposes. The organization faced multiple operational and strategic challenges due to the absence of a structured HR management system. Employee records were maintained in spreadsheets and manually updated, resulting in duplicate entries, inconsistent information, and high dependency on human accuracy. Leave requests were processed through informal communication channels such as email, without automated validation of leave balances or proper approval tracking. This frequently led to approval delays, incorrect leave records, and compliance risks.

Another significant challenge was the lack of a relational database structure. Without normalized data storage and defined relationships between employees, departments, and leave records, maintaining data integrity was difficult. Queries were inefficient, reporting was inconsistent, and there was no optimized mechanism to support performance-aware data retrieval. Management also lacked access to consolidated workforce insights such as leave utilization trends, absenteeism patterns, and department-level workforce analytics. As a result, strategic decisions were often made without reliable data support.

Security was another major concern. There was no role-based authentication mechanism to restrict access to sensitive HR data. Without a structured identity and authorization system, confidential employee information was vulnerable to unauthorized access. Furthermore, the existing system lacked scalability. If the organization expanded from a few hundred employees to several thousand across multiple locations, the manual and loosely structured system would not be sustainable.

To address these challenges, the organization required a secure, scalable, and enterprise-ready HR Management & Workforce Analytics Platform. The system needed to follow layered architecture principles, implement centralized business rule validation, maintain a normalized database structure, provide operational and strategic reporting capabilities, and integrate cloud services only where justified. The ultimate goal was to transform manual HR processes into a structured, data-driven, and future-ready enterprise solution.

1.2 Project Goals and Objectives

The primary goals of this project are defined as follows:

- Develop a scalable, maintainable backend using SOLID principles and layered architecture in C# ASP.NET Core Web API.

- Design a normalized SQL Server database with proper indexing, referential integrity, and performance optimization.

- Build an interactive SPA frontend using HTML, CSS, JavaScript, and TypeScript to provide a seamless user experience.

- Integrate API endpoints with the database to enable full CRUD operations for employee and leave management.

- Generate SSRS reports for leave summaries and employee listings from live database views.

- Create a Power BI Workforce Analytics Dashboard for visual HR insights.
- Document the complete Azure cloud deployment architecture for scalability and security.

Design and implement a SOLID-compliant, layered C# ASP.NET Core 8 Web API backend.

Create a fully normalised (3NF) SQL Server database with appropriate indexing and database views.

Build a Single-Page Application (SPA) frontend using HTML5, CSS3, JavaScript, and TypeScript.

Connect the frontend to the backend API using the browser-native Fetch API.

Integrate SQL Server Reporting Services (SSRS) for structured tabular leave reporting.

Deliver a Power BI Dashboard with DAX measures for real-time workforce analytics.

Conceptualise a Microsoft Azure cloud deployment architecture for production readiness.

Produce comprehensive technical documentation (this report) covering all system components.

1.3 Scope of the System:

The HR Management & Workforce Analytics Platform is designed to provide a structured, scalable, and enterprise-ready solution for managing core human resource operations within a mid-sized organization. The scope of the system primarily focuses on employee data management, leave tracking, business rule validation, reporting, and workforce analytics. The platform aims to replace manual and spreadsheet-based HR processes with a centralized digital system that ensures data accuracy, security, and performance optimization.

At the operational level, the system enables HR personnel to create, update, and manage employee records in a structured format. Each employee is associated with a department, and all records are maintained in a normalized relational database to ensure data integrity and avoid redundancy. The system also manages leave requests by allowing employees' leave data to be recorded and validated according to defined business rules. These validations include duplicate employee prevention, leave balance checks, and approval logic enforcement, ensuring that inconsistent or incorrect HR data is not stored in the system.

From a technical perspective, the scope includes the implementation of a layered architecture using ASP.NET Core Web API, ensuring separation of concerns between models, repositories, services, and controllers. The system incorporates centralized business logic within the service layer to maintain scalability and maintainability. The database design follows normalization principles with proper primary and foreign key relationships, indexing for performance optimization, and reusable views for reporting efficiency.

The system also includes a responsive web-based frontend interface that allows HR users to interact with the application efficiently. The interface supports dynamic updates and smooth navigation to enhance user experience. Operational reporting is handled through SSRS, providing formatted and parameterized reports for leave utilization and absenteeism tracking. For strategic workforce insights, the system integrates Power BI to generate interactive dashboards highlighting key performance indicators such as total employees, department-wise leave trends, and monthly workforce patterns.

Additionally, the scope includes conceptual cloud integration to demonstrate enterprise readiness. Azure Active Directory is considered for secure identity management and role-based authentication, while Azure Functions are proposed for serverless automation scenarios such as leave escalation. However, cloud usage remains minimal and justified, avoiding unnecessary infrastructure complexity while maintaining future scalability potential.

The system is designed to support future expansion, including integration with enterprise HR or ERP systems, increased employee volume, multi-location operations, and enhanced reporting requirements. However, advanced modules such as payroll processing, performance appraisal management, recruitment management, and full-scale cloud deployment are outside the current scope of this project.

Overall, the scope of the system encompasses the digital transformation of essential HR functions, the implementation of enterprise software engineering principles, and the delivery of operational and analytical capabilities that support data-driven workforce management decisions.

Module	Description
| Employee Management | Add, view, update, and delete employee records with department associations. |	| Leave Management | Submit and track leave requests with balance management per employee. |
| Department Management | Manage organizational departments linked to employees. |	| Reporting (SSRS) | Generate formatted leave and employee reports from SQL Views. |
| Analytics (Power BI) | Visual dashboards for total employees, leave trends, and department analysis. |	| Cloud Architecture | Azure-based deployment design for production readiness. |

1.4 Key Stakeholders

- HR Managers – Primary users managing employee records and leave approvals.

- Employees – Submit leave requests and view personal HR data.

- Department Heads – Monitor team attendance and leave trends.

- Management / C-Suite – Access Power BI dashboards for strategic workforce insights.

- IT Administrators – Manage system deployment, security, and Azure infrastructure.

---

2. Frontend & Backend Architecture

Frontend Architecture

The frontend architecture of the HR Management & Workforce Analytics Platform is designed as a lightweight, responsive, and user-friendly presentation layer that interacts seamlessly with the backend through RESTful APIs. The frontend is built using HTML, CSS, Bootstrap, and JavaScript to ensure a structured layout, responsive design, and dynamic user interaction. It follows a client-server architecture where the browser acts as the client and communicates with the backend Web API over HTTP.

The frontend is structured to separate presentation logic from business logic. HTML is used to define the structure of forms, tables, and dashboards, while CSS and Bootstrap provide styling and responsive behavior across devices. JavaScript is used to handle user interactions such as form submissions, validation triggers, and asynchronous API calls using the fetch() method. This enables the application to follow a Single Page Application (SPA)-like behavior, where content updates dynamically without full page reloads, resulting in improved user experience and faster interaction.

When a user submits a form, such as adding a new employee, the frontend captures the input data, performs basic validation, and sends the data as a JSON payload to the backend API endpoint. The frontend does not contain business rules such as duplicate validation or leave balance checks. Instead, it acts as a thin client responsible only for capturing input and displaying results. This architectural choice ensures that critical validation logic remains centralized in the backend, making the system secure, maintainable, and scalable.

The frontend is intentionally kept independent from the database layer. It communicates only with the Web API and does not directly access SQL Server. This separation improves security and allows the frontend to be replaced or extended (for example, with a mobile application) without modifying backend logic. Overall, the frontend architecture focuses on usability, responsiveness, and clean separation from business and data layers.

Backend Architecture

The backend architecture of the HR Management & Workforce Analytics Platform is designed using a layered architecture pattern implemented in ASP.NET Core Web API. This architecture follows the principles of Separation of Concerns and SOLID design principles to ensure modularity, maintainability, and scalability. The backend is divided into four primary layers: Models, Repository Layer, Service Layer, and Controller Layer.

The Model layer defines the data structures that represent entities such as Employees, Departments, and LeaveRequests. These classes correspond directly to database tables and are used for data transfer between layers. By maintaining consistency between models and database schema, the system ensures structural integrity and easier maintenance.

The Repository layer abstracts the data access logic. Instead of allowing controllers or services to directly communicate with the database, repositories handle CRUD operations and data retrieval. This abstraction ensures loose coupling between business logic and data access implementation. Interfaces are used in this layer to apply the Dependency Inversion Principle, allowing easy replacement or extension of data access mechanisms in the future.

The Service layer contains the core business logic of the application. All validation rules, such as preventing duplicate employee emails or validating leave conditions, are centralized in this layer. This ensures that business rules are enforced consistently regardless of how the API is accessed. The service layer communicates with repositories to retrieve or store data, but it does not handle HTTP request or response formatting. By isolating business rules here, the system becomes easier to test and maintain.

The Controller layer serves as the entry point for HTTP requests. Controllers receive requests from the frontend, invoke the appropriate service methods, and return standardized responses. Controllers do not contain business logic; they only coordinate request handling and response formatting. This keeps them lightweight and focused on API exposure.

Dependency Injection is used throughout the backend to manage object lifetimes and dependencies between layers. This promotes loose coupling and enhances testability. The entire backend communicates with SQL Server as the database, where normalized tables, indexes, and views ensure performance optimization and referential integrity.

Overall, the backend architecture ensures that the system is modular, scalable, secure, and aligned with enterprise engineering practices. It supports future enhancements such as ERP integration, authentication via Azure AD, and automation through Azure Functions without requiring structural redesign.

2.1 Technology Stack Overview

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	SPA interface, dynamic form handling, responsive UI interactions
Backend	C# ASP.NET Core Web API (.NET 8)	RESTful API development, business logic implementation, DI container, Swagger API testing
Database	SQL Server 2022	Relational data storage, normalized schema, views, indexes, stored procedures
Reporting	SQL Server Reporting Services (SSRS)	Paginated operational reports generated from SQL views
Analytics	Power BI Desktop	Interactive dashboards, KPI tracking, workforce trend analysis
Cloud	Microsoft Azure	Scalable hosting, managed database services, identity management (Azure AD)
IDE / Tools	Visual Studio 2022, SQL Server Management Studio (SSMS)	Application development, debugging, database creation and management

2.2 Overall System Architecture

The system follows a classic N-Tier / Layered Architecture pattern. This design separates concerns across distinct layers, ensuring that each layer has a single responsibility and can be independently developed, tested, and maintained. The HR Management & Workforce Analytics Platform follows a structured multi-layered architecture designed to ensure scalability, maintainability, and enterprise readiness. The system is built using a client-server model where the frontend presentation layer communicates with a backend RESTful API, which in turn interacts with a relational database. Additional reporting and analytics tools consume processed data to generate operational and strategic insights.

At the highest level, the system consists of five major architectural components: the Presentation Layer (Frontend), Application Layer (Backend Web API), Business Logic Layer (Service Layer), Data Access Layer (Repository Layer), and the Database Layer (SQL Server). These components work together to ensure separation of concerns and modular system design.

The Presentation Layer is developed using HTML, CSS, Bootstrap, and JavaScript. It runs in the user's browser and acts as the interface through which HR personnel interact with the system. When a user submits data, such as adding a new employee or viewing leave information, the frontend captures the input and sends it as a structured JSON request to the backend API using HTTP methods. The frontend does not contain business validation logic; instead, it focuses solely on capturing input and displaying responses. This design ensures security and centralized rule enforcement.

The Application Layer is implemented using ASP.NET Core Web API (.NET 8). This layer exposes RESTful endpoints that handle HTTP requests from the frontend. The controller classes receive requests, validate request formats, and forward them to the Service Layer. Controllers do not implement business rules; they only coordinate communication between the client and the business logic.

The Business Logic Layer (Service Layer) contains the core rules of the system. For example, it validates duplicate employee emails, enforces leave approval logic, and ensures that only valid data is passed to the database. Centralizing business logic in this layer ensures consistency, testability, and scalability. If new rules are introduced in the future, they can be added here without modifying other layers.

The Data Access Layer (Repository Layer) abstracts communication with the database. Instead of directly writing database queries inside controllers or services, the repository layer handles data retrieval and persistence operations. Interfaces are used to promote loose coupling and to follow the Dependency Inversion Principle. This design improves maintainability and allows easier replacement of database access mechanisms if required.

The Database Layer is implemented using SQL Server 2022. The database follows normalization principles (Third Normal Form) to eliminate redundancy and maintain referential integrity. Tables such as Departments, Employees, and LeaveRequests are connected through primary and foreign keys. Indexes are created for performance optimization, and views are designed to simplify reporting queries.

Beyond core transaction processing, the architecture also integrates Reporting and Analytics components. SQL Server Reporting Services (SSRS) consumes data from database views to generate structured, printable reports such as leave utilization summaries. Power BI connects to SQL Server to build interactive dashboards that visualize workforce trends, department-level analysis, and leave patterns for strategic decision-making.

For enterprise readiness, the architecture includes conceptual cloud integration. Azure Active Directory can be integrated for secure identity management and role-based authentication. Azure Functions can be used for serverless automation tasks such as leave escalation workflows. However, cloud usage remains minimal and justified to avoid unnecessary complexity.

Overall, the system architecture ensures clean separation between presentation, business logic, data access, and storage layers. It supports scalability from small organizations to large enterprises, allows easy integration with ERP systems, and maintains security and performance optimization throughout the application lifecycle.

Architecture Flow:

PRESENTATION LAYER

HTML / CSS / JavaScript / TypeScript (SPA)

↓ HTTP Requests (JSON)

API / CONTROLLER LAYER

ASP.NET Core Web API – Routing, HTTP Verbs, Swagger

↓ Method Calls

SERVICE / BUSINESS LAYER

EmployeeService, LeaveService – Business Rules, Validations

↓ Interface Calls (DIP)

REPOSITORY / DATA LAYER

IEmployeeRepository, EmployeeRepository – Data Access

↓ SQL Queries

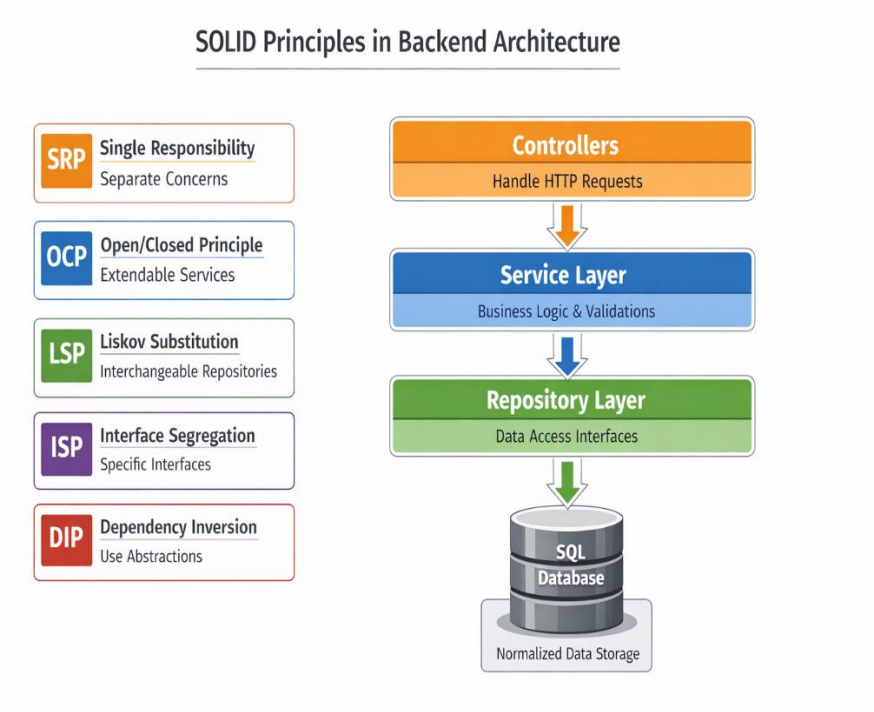
DATABASE LAYER

SQL Server – Tables, Views, Indexes, Referential Integrity

2.3 Backend Architecture – SOLID Principles Applied

Principle	Abbreviation	Implementation in This Project
Single Responsibility	SRP	Each class handles one concern: EmployeeRepository manages data access; EmployeeService enforces business rules.
Open/Closed	OCP	New employee types or leave policies can be added by extending classes, not modifying existing ones.
Liskov Substitution	LSP	Any concrete repository (EmployeeRepository) can be substituted for its interface (IEmployeeRepository).
Interface Segregation	ISP	Interfaces are minimal: IEmployeeRepository exposes only GetAll, GetById, and Add.
Dependency Inversion	DIP	EmployeeService depends on the IEmployeeRepository abstraction, not the concrete EmployeeRepository implementation.

2.4 Frontend Architecture – SPA Design



The frontend follows a Single Page Application pattern, where the entire application loads once and all subsequent interactions happen dynamically without full page reloads. This provides a fast, responsive, app-like experience.

Frontend Folder Structure:

HRFrontend

├── index.html ← Main HTML shell (single page)

├── style.css ← Global styles, responsive layout

├── app.js ← Runtime JavaScript, API calls, DOM manipulation

└── app.ts ← TypeScript source, type definitions, interfaces

TypeScript Interface Definitions:

typescript

```
interface Employee {
    employeeId: number;
    name: string;
    email: string;
    departmentId: number;
    leaveBalance: number;
}

interface LeaveRequest {
    leaveRequestId: number;
    employeeId: number;
    daysRequested: number;
    status: 'Pending' | 'Approved' | 'Rejected';
}

interface Department {
```

```
  departmentId: number;
  departmentName: string;
}
```

---

3. Component Breakdown & API Design

3.1 Backend Component Breakdown

3.1.1 Models Layer

Models represent the core data structures shared between the API, service layer, and database. They act as Data Transfer Objects (DTOs) within the application. The Models Layer represents the foundational data structure of the HR Management & Workforce Analytics Platform. This layer defines the core entities of the system such as Employee, Department, and LeaveRequest. Each model class corresponds directly to a database table and acts as a blueprint for the data that flows between different layers of the application. The primary purpose of the Models layer is to define the shape, structure, and properties of the business data in a clean and organized manner.

In this project, the Employee model contains properties such as EmployeeId, Name, Email, DepartmentId, and LeaveBalance. These properties directly map to columns in the Employees table in SQL Server. Similarly, the LeaveRequest model represents leave-related data including LeaveRequestId, EmployeeId, DaysRequested, and Status. By keeping these models aligned with the database schema, the system ensures consistency between the application layer and the data layer.

The Models layer plays a critical role in maintaining type safety and structured data exchange. When the frontend sends data to the backend API in JSON format, the ASP.NET Core framework automatically binds that data to the appropriate model class. This process, known as model binding, ensures that only correctly structured data enters the business logic layer. If invalid or incomplete data is provided, validation mechanisms can prevent incorrect processing before reaching the database.

Another important function of the Models layer is to act as Data Transfer Objects (DTOs) between layers. Controllers receive models from the client, pass them to the Service layer for validation, and eventually to the Repository layer for persistence. By keeping models simple and focused only on data representation, the system follows the Single Responsibility Principle. The models do not contain business logic or database access code; they purely represent structured data.

Additionally, maintaining models separately from business logic allows the system to evolve easily. If new fields are required, such as employee designation or leave type, they can be added to the model and database without affecting the architectural structure of other layers. This separation enhances maintainability, scalability, and clarity of the overall backend design.

In summary, the Models layer serves as the structural backbone of the HR Management System. It ensures clear representation of business entities, supports smooth data transfer across layers, maintains alignment with the database schema, and upholds clean architecture principles within the backend system.

Employee.cs:

csharp

```
public class Employee
{
    public int EmployeeId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public int DepartmentId { get; set; }
    public int LeaveBalance { get; set; }
}
```

LeaveRequest.cs:

csharp

```
public class LeaveRequest
{
    public int LeaveRequestId { get; set; }
```

```
    public int EmployeeId { get; set; }
    public int DaysRequested { get; set; }
    public string Status { get; set; }
}
```

3.1.2 Repository Layer (Data Access Layer)

The Repository Layer abstracts all database interactions. By programming to interfaces (IEmployeeRepository), the service layer remains completely independent of the underlying data store, enabling easy unit testing and future database migration.

The Repository Layer in the HR Management & Workforce Analytics Platform is responsible for handling all data access operations between the application and the SQL Server database. It acts as an abstraction layer that separates business logic from direct database interaction. Instead of allowing controllers or services to write SQL queries or manipulate database connections directly, the repository layer centralizes all data-related operations, ensuring clean architecture and maintainability.

In this project, repository classes such as EmployeeRepository implement corresponding interfaces like IEmployeeRepository. These repositories provide methods such as GetAll(), GetById(), and Add(), which encapsulate the logic required to retrieve and store employee data. By defining these methods inside an interface, the system applies the Dependency Inversion Principle, allowing higher-level modules (like the Service layer) to depend on abstractions rather than concrete implementations.

The primary purpose of the repository layer is to manage Create, Read, Update, and Delete (CRUD) operations. When a service requires employee data, it calls the repository instead of directly querying the database. The repository then communicates with the database, executes the required query, and returns structured data back to the service layer. This approach ensures that the database access logic remains centralized and reusable across the system.

Another significant advantage of this design is loose coupling. If the organization decides to switch from in-memory storage to Entity Framework Core or from SQL Server to another database system in the future, only the repository implementation needs to be modified. The rest of the application remains unaffected because it depends on the repository interface rather than the concrete class. This makes the system highly adaptable and scalable.

The repository layer also improves testability. Since services depend on interfaces, mock implementations of repositories can be injected during unit testing. This allows developers to test business logic independently without requiring an actual database connection. Such separation enhances reliability and promotes clean software engineering practices.

In summary, the Repository Layer ensures organized data access, promotes abstraction, enhances maintainability, and supports scalability within the HR Management System. By isolating database operations from business logic, the system adheres to layered architecture principles and ensures a clean, enterprise-ready backend design.

csharp

```
// Interface – defines the contract
public interface IEmployeeRepository
{
    List<Employee> GetAll();
    Employee GetById(int id);
    void Add(Employee employee);
    void Update(Employee employee);
    void Delete(int id);
}

// Concrete Implementation
public class EmployeeRepository : IEmployeeRepository
{
    private static List<Employee> employees = new();

    public List<Employee> GetAll() => employees;

    public Employee GetById(int id)
        => employees.FirstOrDefault(e => e.EmployeeId == id);

    public void Add(Employee employee)
```

```
        => employees.Add(employee);

    public void Update(Employee employee)
    {
        var existing = GetById(employee.EmployeeId);
        if (existing != null)
        {
            existing.Name = employee.Name;
            existing.Email = employee.Email;
            existing.DepartmentId = employee.DepartmentId;
            existing.LeaveBalance = employee.LeaveBalance;
        }
    }

    public void Delete(int id)
        => employees.RemoveAll(e => e.EmployeeId == id);
}
```

3.1.3 Service Layer (Business Logic Layer)

The Service Layer contains all business rules and validation logic. It orchestrates calls to the repository and enforces constraints such as duplicate email prevention, leave balance checks, and status transitions.

The Service Layer in the HR Management & Workforce Analytics Platform is responsible for implementing the core business logic of the system. It acts as an intermediary between the Controller Layer and the Repository Layer, ensuring that all business rules are validated before data is stored or retrieved from the database. This layer is the most critical component of the backend architecture because it enforces organizational policies and maintains data integrity.

In this project, the EmployeeService class contains validation logic such as preventing duplicate employee email entries. When a new employee is added, the service layer first retrieves existing records from the repository and checks whether an employee with the same email already exists. If a duplicate is found, the service throws an exception to prevent invalid data from being stored. This ensures that business rules are centralized and consistently enforced across the application.

The Service Layer does not handle HTTP requests or direct database queries. Instead, it focuses solely on business operations such as validation, conditional logic, and process control. For example, in the leave management process, business rules such as leave approval logic, leave balance validation, and status updates would be implemented in this layer. By isolating business logic from both the user interface and data access layers, the system adheres to the Single Responsibility Principle and ensures maintainability.

Another important aspect of the Service Layer is scalability. If the organization introduces new policies, such as advanced leave categories or department-based approval hierarchies, these changes can be implemented within the service classes without modifying controllers or repositories. This design follows the Open/Closed Principle, allowing the system to extend functionality without altering stable core components.

Overall, the Service Layer acts as the brain of the HR Management System. It ensures that all business rules are applied consistently, maintains data correctness, enhances system scalability, and supports clean separation of concerns within the backend architecture.

csharp

```
public class EmployeeService
{
    private readonly IEmployeeRepository _repository;

    public EmployeeService(IEmployeeRepository repository)
        => _repository = repository;  // DIP – inject abstraction

    public void AddEmployee(Employee employee)
    {
        // Business Rule: Duplicate email check
        if (_repository.GetAll().Any(e => e.Email == employee.Email))
```

```
            throw new Exception("Duplicate Email Not Allowed");


        // Business Rule: Leave balance must be non-negative
        if (employee.LeaveBalance < 0)
            throw new ArgumentException("Leave balance cannot be negative.");


        _repository.Add(employee);
    }


    public List<Employee> GetAllEmployees()
        => _repository.GetAll();


    public Employee GetEmployeeById(int id)
        => _repository.GetById(id)
          ?? throw new Exception($"Employee with ID {id} not found.");
}
```

3.1.4 Controller Layer:

The Controller Layer serves as the entry point of the backend system. It is responsible for handling HTTP requests coming from the frontend and returning appropriate HTTP responses. In the HR Management & Workforce Analytics Platform, controllers expose RESTful API endpoints that allow the frontend to perform operations such as adding employees, retrieving records, and managing leave data.

Each controller is decorated with attributes such as [ApiController] and [Route("api/[controller]")], which define how incoming requests are mapped to specific methods. For example, the EmployeeController contains an HTTP POST method that receives employee data from the client. The controller then forwards this data to the Service Layer for validation and processing. Once the service completes the operation, the controller returns a standardized HTTP response such as Ok() or an error message.

A key design principle followed in this layer is that controllers do not contain business logic. Their sole responsibility is request handling and response formatting. By keeping controllers lightweight, the system ensures better maintainability and adherence to the Single Responsibility Principle. If validation or business rules were implemented directly inside controllers, the system would become tightly coupled and harder to maintain.

The Controller Layer also integrates with Swagger (OpenAPI) for API documentation and testing. This allows developers and testers to verify API endpoints without relying on the frontend. Such integration enhances transparency and supports enterprise-level development practices.

In summary, the Controller Layer acts as the communication bridge between the frontend and backend logic. It receives client requests, delegates processing to the Service Layer, and returns structured responses. This design ensures clean API structure, scalability, and compliance with RESTful architectural standards.

csharp

```
[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
    private readonly EmployeeService _service;

    public EmployeeController(EmployeeService service)
        => _service = service;

    [HttpGet]
    public IActionResult GetAll()
        => Ok(_service.GetAllEmployees());

    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
```

```
        var emp = _service.GetEmployeeById(id);
        return emp != null ? Ok(emp) : NotFound();
    }

    [HttpPost]
    public IActionResult Add(Employee employee)
    {
        _service.AddEmployee(employee);
        return Ok("Employee Added Successfully");
    }
}
```

3.2 API Design and Endpoints

All endpoints return JSON-formatted responses and are documented via Swagger / OpenAPI at: http://localhost:5000/swagger

Employee API Endpoints

Method	Endpoint	Status Code	Description
GET	/api/employee	200 OK	Retrieve all employees
GET	/api/employee {id}	200 OK / 404	Get employee by ID
POST	/api/employee	200 OK / 400	Add a new employee
PUT	/api/employee/{id}	200 OK / 404	Update employee details
DELETE	/api/employee/{id}	200 OK / 404	Remove employee record

Leave Request API Endpoints

Method	Endpoint	Status Code	Description
GET	/api/leave	200 OK	Retrieve all leave requests
GET	/api/leave/{id}	200 OK / 404	Get leave request by ID
POST	/api/leave	200 OK / 400	Submit a new leave request
PUT	/api/leave/{id}/approve	200 OK	Approve a leave request
PUT	/api/leave/{id}/reject	200 OK	Reject a leave request

3.3 Dependency Injection Configuration

csharp

```
var builder = WebApplication.CreateBuilder(args);

// Register Repository (Singleton – one instance for app lifetime)
builder.Services.AddSingleton<IEmployeeRepository, EmployeeRepository>();
builder.Services.AddSingleton<ILeaveRepository, LeaveRepository>();

// Register Services (Scoped – new instance per HTTP request)
builder.Services.AddScoped<EmployeeService>();
builder.Services.AddScoped<LeaveService>();

// Enable Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
app.UseSwagger();
```

```
app.UseSwaggerUI();

app.MapControllers();

app.Run();
```

3.4 Exception Handling

Custom exceptions are used across the service layer to provide meaningful error feedback. Exception handling in the HR Management & Workforce Analytics Platform is implemented to ensure system reliability, user-friendly error responses, and proper error logging. In any enterprise application, unexpected situations such as duplicate data entry, invalid inputs, missing records, or database failures can occur. Without structured exception handling, such issues can cause system crashes, data corruption, or poor user experience. Therefore, a centralized and structured exception handling mechanism is essential.

In this project, exception handling is primarily implemented within the Service Layer. Business rule violations such as duplicate employee email entries or invalid leave operations are detected in the service classes. For example, when adding a new employee, the system checks whether an employee with the same email already exists. If a duplicate is found, the service throws an exception with a meaningful message such as "Duplicate Email Not Allowed." This prevents invalid data from being stored in the database and ensures data integrity.

The Controller Layer is responsible for capturing these exceptions and returning appropriate HTTP status codes. For instance, if a duplicate employee is detected, the API can return a 400 Bad Request response with a descriptive error message. If a requested employee record does not exist, the API can return a 404 Not Found response. This ensures that clients interacting with the API receive clear and standardized feedback.

In enterprise-ready implementations, global exception handling middleware can be configured in ASP.NET Core. This middleware intercepts unhandled exceptions across the entire application and returns structured JSON error responses. It also prevents exposure of sensitive system details, such as stack traces, to end users. Instead, it provides clean error messages while optionally logging detailed information internally for debugging purposes.

Exception handling also improves maintainability and debugging efficiency. By centralizing error responses and handling logic, developers can track issues systematically. Additionally, logging mechanisms can be integrated to store error details for future analysis and system improvement.

Overall, exception handling in the HR Management System ensures system stability, protects data integrity, improves user experience, and aligns with enterprise software development best practices.

csharp

```
// Custom Exception
public class DuplicateEmailException : Exception
{
    public DuplicateEmailException(string message) : base(message) { }
}

// Global Middleware in Program.cs
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        context.Response.ContentType = "application/json";
        context.Response.StatusCode = 400;
        await context.Response.WriteAsync(
            JsonSerializer.Serialize(new { error = "An error occurred." }));
    });
});
```

---

4. Database Design & Storage Optimization

4.1 Database Design Principles

The HRManagementDB database is designed following Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. Each table has a clear primary key, all non-key columns are fully functionally dependent on the primary key, and there are no transitive dependencies. The database design for the HR Management & Workforce Analytics Platform is structured to ensure data integrity, consistency, scalability, and performance optimization. Since the database serves as the foundation of the system, careful planning and adherence to relational database design principles were essential to build a stable and enterprise-ready solution.

The first key principle applied in this project is **Normalization**, specifically up to the Third Normal Form (3NF). Normalization was implemented to eliminate data redundancy and prevent anomalies during insert, update, and delete operations. Employee details, department information, and leave requests are stored in separate tables with defined relationships. This ensures that repeated data is not unnecessarily duplicated across the database. For example, department names are stored only once in the Departments table and referenced in the Employees table using a foreign key.

Another important principle followed is **Referential Integrity**. Primary keys and foreign keys are defined to maintain logical relationships between tables. The Employees table contains a DepartmentId that references the Departments table, and the LeaveRequests table references EmployeeId from the Employees table. This ensures that a leave request cannot exist without a valid employee record, thereby maintaining relational consistency and preventing orphan records.

The design also incorporates **Indexing for Performance Optimization**. Since department-based queries and employee lookups are common operations in HR systems, indexes are created on frequently queried columns such as DepartmentId. This significantly improves query execution speed and ensures efficient data retrieval, especially as the number of employee records grows.

To simplify reporting and analytics, **Database Views** are created. For example, the vw_LeaveReport view joins Employees, Departments, and LeaveRequests into a structured dataset suitable for SSRS and Power BI reporting. This approach reduces complexity in reporting tools and improves reusability of SQL logic.

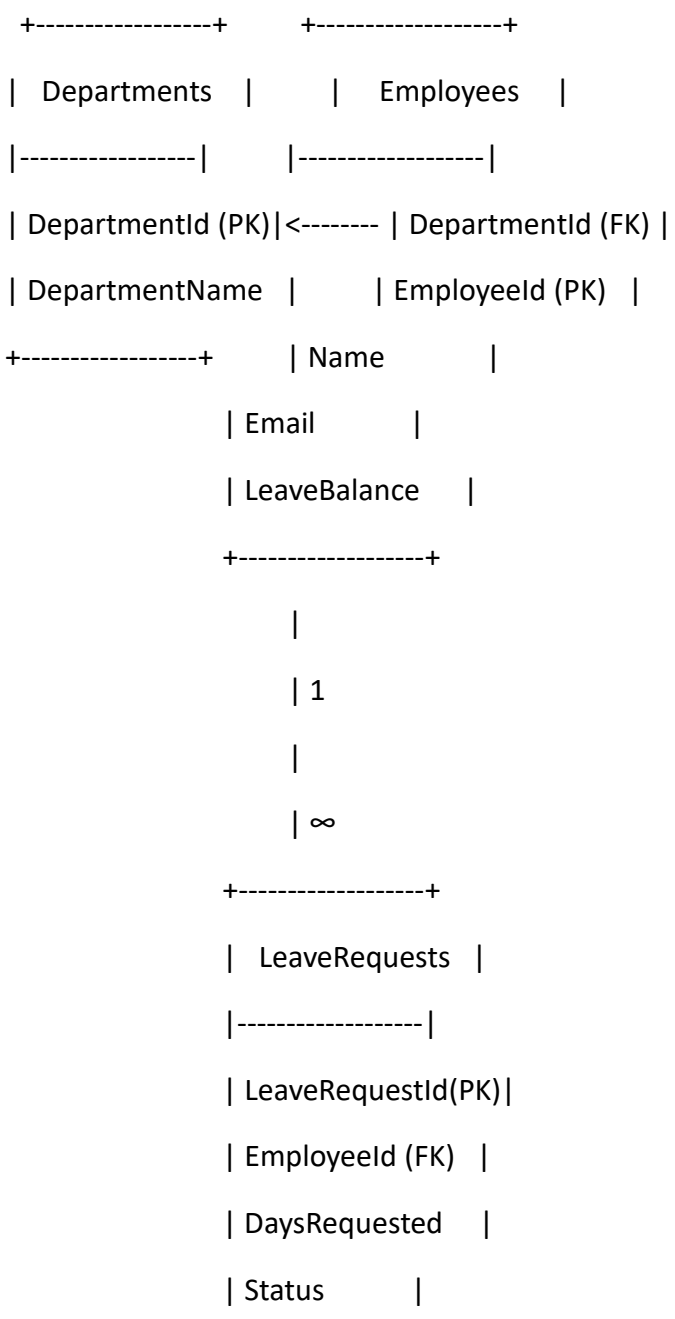
Another principle followed is **Data Consistency through Constraints**. Unique constraints are applied to fields such as Email to prevent duplicate employee entries. Proper data types are selected for each column to ensure accuracy and storage efficiency. For instance, integer types are used for IDs and leave days, while varchar fields are used for textual data.

The database is also designed with **Scalability in Mind**. By separating master data (Departments), transactional data (LeaveRequests), and entity data (Employees), the

structure supports expansion without redesign. If additional modules such as payroll or performance management are introduced in the future, they can be integrated without disrupting the existing schema.

Overall, the database design principles applied in this HR Management System ensure structured data storage, strong relational integrity, optimized performance, reduced redundancy, and enterprise-level scalability. This foundation supports reliable backend operations, accurate reporting, and future system growth.

4.2 Entity-Relationship Diagram (ERD)



+--------------------+

Relationships

Departments (1) ———— (N) Employees

One department has many employees.

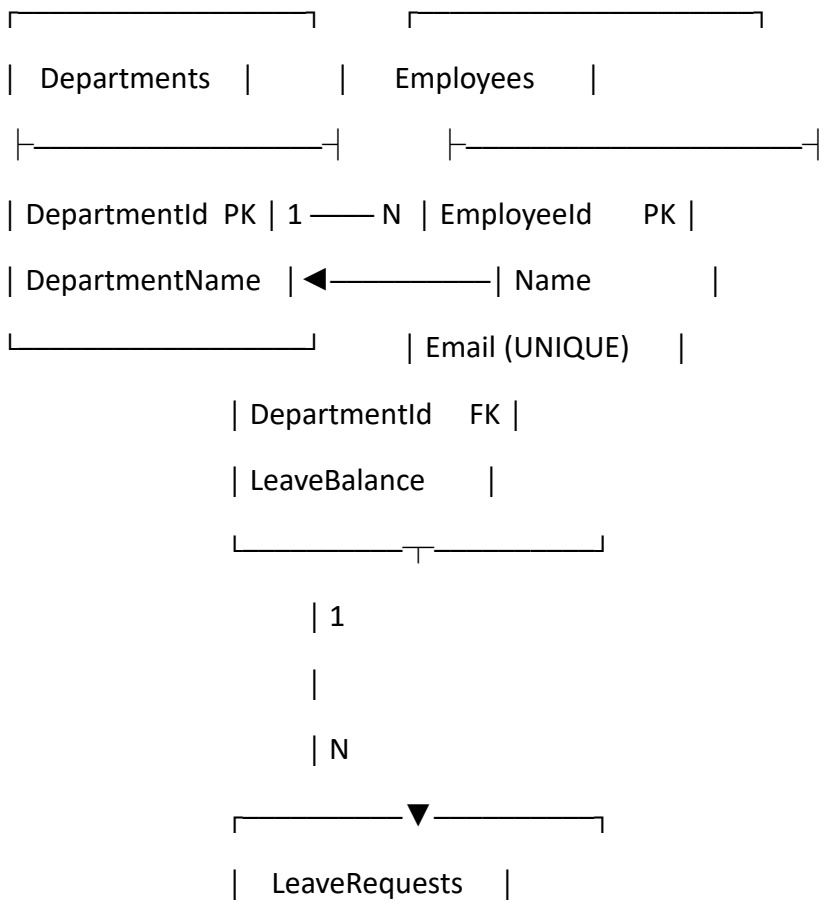
DepartmentId in Employees references DepartmentId in Departments.

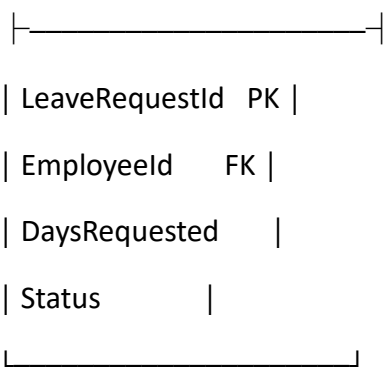
Employees (1) ———— (N) LeaveRequests

One employee can have many leave requests.

EmployeeId in LeaveRequests references EmployeeId in Employees.

ERD Text Diagram:





4.3 Database Creation Scripts

sql

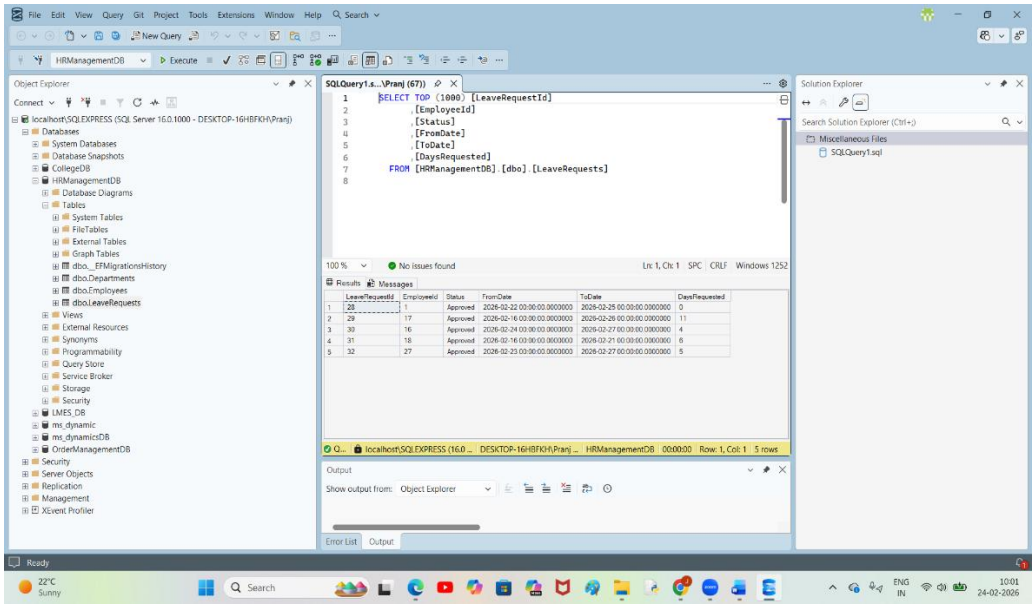
```
CREATE DATABASE HRManagementDB;
GO
USE HRManagementDB;
GO

-- DEPARTMENTS TABLE
CREATE TABLE Departments (
    DepartmentId   INT PRIMARY KEY IDENTITY(1,1),
    DepartmentName VARCHAR(100) NOT NULL
);

-- EMPLOYEES TABLE (References Departments)
CREATE TABLE Employees (
    EmployeeId   INT PRIMARY KEY IDENTITY(1,1),
    Name         VARCHAR(100) NOT NULL,
    Email        VARCHAR(100) NOT NULL UNIQUE,
    DepartmentId INT NOT NULL,
    LeaveBalance INT DEFAULT 20,
```

```
    CONSTRAINT FK_Emp_Dept FOREIGN KEY (DepartmentId)
        REFERENCES Departments(DepartmentId)
        ON UPDATE CASCADE
);

-- LEAVE REQUESTS TABLE (References Employees)
CREATE TABLE LeaveRequests (
    LeaveRequestId INT PRIMARY KEY IDENTITY(1,1),
    EmployeeId     INT NOT NULL,
    DaysRequested  INT NOT NULL CHECK (DaysRequested > 0),
    Status         VARCHAR(50) DEFAULT 'Pending',
    RequestDate    DATE DEFAULT GETDATE(),
    CONSTRAINT FK_Leave_Emp FOREIGN KEY (EmployeeId)
        REFERENCES Employees(EmployeeId)
        ON DELETE CASCADE
);
```



4.4 Storage Optimization Techniques

4.4.1 Indexing Strategy

Indexes dramatically improve query performance by allowing SQL Server to locate rows without full table scans. The indexing strategy in the HR Management & Workforce Analytics Platform is designed to improve query performance, optimize data retrieval speed, and ensure efficient handling of growing datasets. Since HR systems frequently perform search, filter, and join operations across multiple tables, proper indexing is critical to maintaining system responsiveness and scalability.

In this project, indexes are applied to columns that are frequently used in search conditions, join operations, and filtering queries. One of the primary indexing implementations is on the DepartmentId column in the Employees table. Since employee data is commonly retrieved department-wise for reporting and analytics purposes, indexing this column significantly reduces query execution time when filtering employees by department.

Additionally, the Email column in the Employees table is defined with a unique constraint. This not only enforces data integrity by preventing duplicate entries but also automatically creates a unique index on the column. As a result, lookups based on employee email—such as validation during employee creation—are performed efficiently.

Foreign key columns, such as EmployeeId in the LeaveRequests table, are also strategically indexed. Since join operations between Employees and LeaveRequests are common in reports and dashboards, indexing these foreign key fields improves join performance and reduces the time required to generate leave-related summaries.

The indexing strategy follows a performance-aware design approach. Indexes are applied selectively to avoid unnecessary overhead. While indexes improve read performance, excessive indexing can slow down insert and update operations. Therefore, only frequently queried and relational columns are indexed, maintaining a balance between read efficiency and write performance.

In the reporting context, since SSRS and Power BI rely on views that join multiple tables, indexing key relational fields ensures that report generation remains fast even as the volume of employee and leave data increases. This makes the system scalable and suitable for enterprise environments where thousands of employee records may exist.

Overall, the indexing strategy in this HR Management System enhances database performance, supports efficient query execution, improves reporting speed, and ensures that the system remains responsive as data volume grows. It reflects a well-planned database optimization approach aligned with enterprise-level best practices.

sql

```
-- Index on DepartmentId for employee lookup by department
CREATE INDEX idx_emp_dept
ON Employees(DepartmentId);

-- Unique index on Email for fast duplicate checking
CREATE UNIQUE INDEX idx_emp_email
ON Employees(Email);

-- Index on EmployeeId for fast leave lookup
CREATE INDEX idx_leave_emp
ON LeaveRequests(EmployeeId);

-- Composite index for filtering leaves by status
CREATE INDEX idx_leave_status
ON LeaveRequests(EmployeeId, Status);
```

4.4.2 Database View

A SQL View named vw_LeaveReport encapsulates the complex JOIN logic. This view is reused by SSRS reports, API queries, and Power BI connections, eliminating code duplication and centralizing query maintenance.

sql

```
CREATE VIEW vw_LeaveReport AS
SELECT
    e.EmployeeId,
```

```
    e.Name          AS EmployeeName,
    d.DepartmentName,
    l.LeaveRequestId,
    l.DaysRequested,
    l.Status,
    l.RequestDate
FROM Employees e
JOIN Departments d ON e.DepartmentId = d.DepartmentId
JOIN LeaveRequests l ON e.EmployeeId = l.EmployeeId;
GO

-- Sample Usage
SELECT * FROM vw_LeaveReport WHERE Status = 'Pending';
SELECT * FROM vw_LeaveReport ORDER BY DepartmentName;
```

4.4.3 Normalization Summary

Normalization in the HR Management & Workforce Analytics Platform is implemented to ensure data consistency, eliminate redundancy, and maintain relational integrity within the SQL Server database. Since the system manages employee records, department data, and leave transactions, a structured relational model is essential to prevent duplication and anomalies. The database design follows normalization principles up to the Third Normal Form (3NF), ensuring a stable and scalable data structure.

The First Normal Form (1NF) is achieved by ensuring that all tables contain atomic values, meaning each column stores only a single value and there are no repeating groups or multi-valued attributes. For example, employee details such as Name, Email, and LeaveBalance are stored in separate columns with single, indivisible values. Each record in the Employees table represents one employee only, with a unique primary key (EmployeeId), ensuring row-level uniqueness.

The Second Normal Form (2NF) is satisfied by removing partial dependencies. In this project, each table has a single-column primary key, and all non-key attributes depend entirely on that primary key. For instance, in the Employees table, attributes such as Name, Email, and

DepartmentId depend solely on EmployeeId. There are no composite keys causing partial dependency issues, which ensures that each piece of data belongs logically to its entity.

The Third Normal Form (3NF) is achieved by eliminating transitive dependencies. Non-key attributes depend only on the primary key and not on other non-key attributes. For example, department details such as DepartmentName are stored in the Departments table rather than being repeated in the Employees table. Employees reference departments through a foreign key (DepartmentId), preventing duplication of department names across employee records. This separation ensures that updating a department name requires modification in only one location.

By following normalization up to 3NF, the database prevents common anomalies. Insert anomalies are avoided because department records can be created independently before assigning employees. Update anomalies are prevented because department or employee information is stored in a single location. Delete anomalies are avoided since removing a leave request does not remove employee or department data.

Overall, normalization in the HR Management System ensures structured data organization, improved data integrity, reduced redundancy, easier maintenance, and enhanced scalability. This foundation supports accurate reporting, efficient query performance, and reliable backend operations, making the system suitable for enterprise-level deployment.

---

5. SSRS Reporting

5.1 Overview of SSRS

SQL Server Reporting Services (SSRS) is a server-based reporting platform that allows creation of formatted, paginated reports from SQL Server data. In this project, SSRS is used to generate leave summary reports and employee listings that can be exported to PDF, Excel, or printed directly. SQL Server Reporting Services (SSRS) is used in the HR Management & Workforce Analytics Platform to generate structured, paginated, and printable reports directly from the SQL Server database. In an enterprise HR system, operational reporting is critical for monitoring employee activities, leave usage, and departmental performance. SSRS provides a reliable platform for creating formal reports that can be exported to PDF, Excel, or printed for administrative purposes.

In this project, SSRS is connected to the HRManagementDB database as the primary data source. The reports are designed using SQL views, particularly the vw_LeaveReport view,

which joins Employees, Departments, and LeaveRequests into a unified dataset. This approach simplifies reporting logic and ensures consistency between operational reports and database records. By using views instead of complex inline queries, report maintenance becomes easier and more structured.

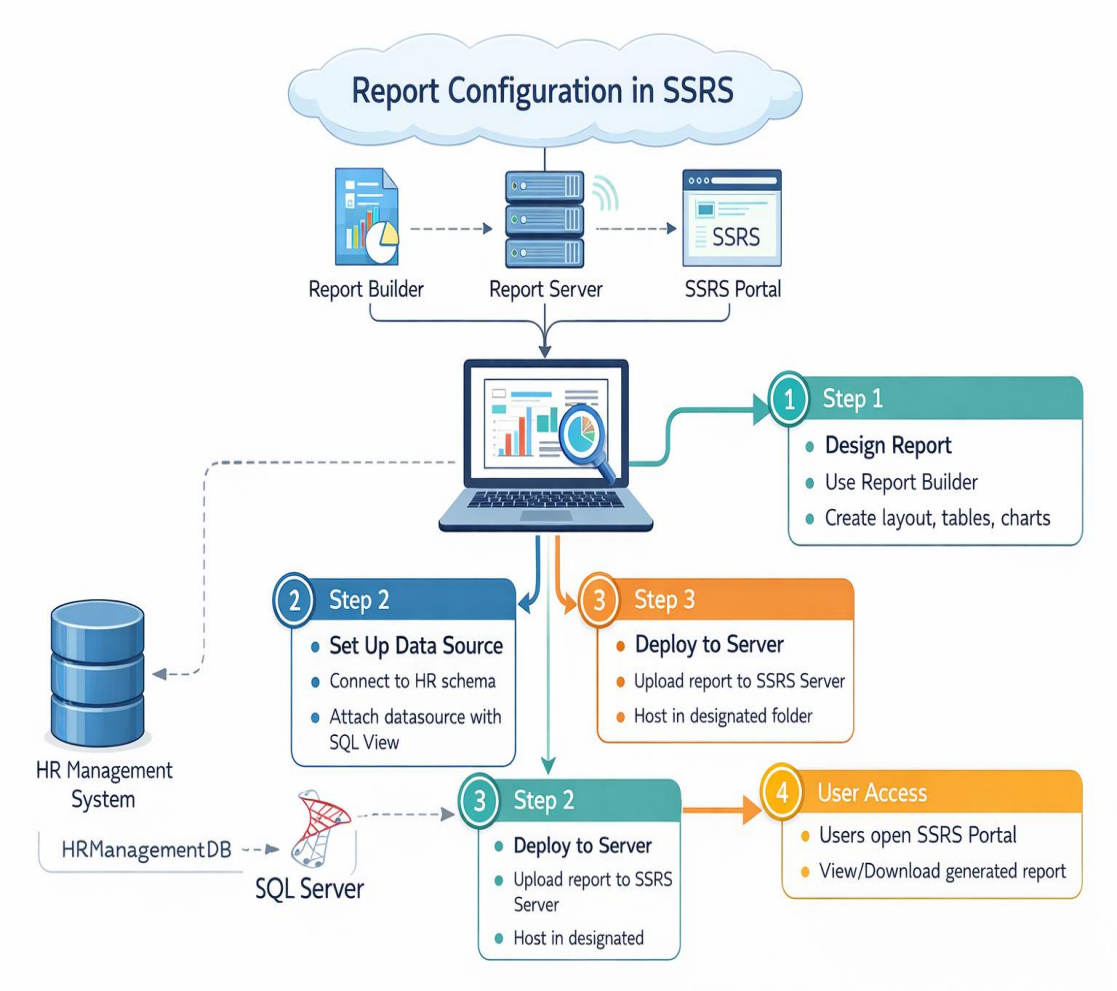
The primary objective of using SSRS in this system is to provide operational-level insights to HR managers. Reports such as leave utilization summaries, department-wise leave analysis, and employee leave history can be generated in a clean tabular format. These reports support day-to-day HR decision-making, audit compliance, and documentation requirements. Since SSRS reports are paginated, they are ideal for formal documentation where structured layouts, headers, footers, and grouped data are required.

SSRS also supports parameterized reporting. For example, HR managers can filter reports by department, date range, or employee ID. This dynamic filtering enhances usability and allows focused analysis without modifying the underlying query. The ability to customize reports based on input parameters makes the reporting system flexible and scalable.

Another advantage of SSRS in this project is its seamless integration with SQL Server. Since the database is already structured and normalized, SSRS can efficiently retrieve data using indexed columns and predefined views. This ensures optimal performance even as the dataset grows.

Overall, SSRS in the HR Management System provides reliable, structured, and enterprise-ready reporting capabilities. It complements the transactional backend by converting raw data into meaningful operational reports that support HR administration, compliance, and workforce monitoring.

5.2 Report Configuration



5.3 Report Design Steps

Step 1: Launch Visual Studio 2022 and create a new Report Server Project.

Step 2: Add a Shared Data Source pointing to HRManagementDB using Windows Authentication.

Step 3: Add a new Report (.rdl file) using the Report Wizard.

Step 4: Create a Dataset using the query

sql

SELECT * FROM vw_LeaveReport;

Step 5: Design the report layout by inserting a Table control and mapping the following fields — EmployeeName, DepartmentName, DaysRequested, Status, RequestDate.

Step 6: Add a report header with the title "HR Leave Summary Report" and insert the current date expression:

Step 7: Add grouping by DepartmentName to display department-wise leave totals with subtotals.

Step 8: Preview the report in Visual Studio, verify data, capture a screenshot for documentation, and deploy to the Report Server.

---

6. Power BI Workforce Analytics Dashboard

6.1 Purpose and Overview

The Power BI Workforce Analytics Dashboard provides management with a visual, interactive overview of the organization's HR data. It transforms raw database records into meaningful insights, enabling data-driven decisions on staffing, leave management, and departmental performance. The Power BI Workforce Analytics Dashboard in the HR Management & Workforce Analytics Platform is designed to transform raw HR data into meaningful, visual, and interactive insights that support strategic decision-making. While the backend system manages employee records and leave transactions, and SSRS provides structured operational reports, Power BI serves as the analytical layer of the system. Its primary purpose is to provide management-level visibility into workforce trends, performance patterns, and leave utilization metrics in a dynamic and user-friendly format.

The dashboard connects directly to the HRManagementDB database and retrieves data from normalized tables and reporting views. By integrating employee, department, and leave request data, Power BI generates consolidated insights that help leadership understand workforce behavior over time. Instead of reviewing static reports, decision-makers can interact with filters, slicers, and visual charts to analyze data based on departments, time periods, or specific employee groups.

One of the key objectives of the dashboard is to monitor leave utilization trends across departments. By visualizing leave data through bar charts, line graphs, and KPI cards, HR

managers can identify departments with high absenteeism, seasonal leave patterns, and potential workforce shortages. This allows proactive planning and better resource allocation.

The dashboard also includes high-level metrics such as total number of employees, total leave requests, average leave days per employee, and department-wise leave distribution. These KPIs provide a quick summary of workforce health and operational efficiency. Since the visuals update automatically when data changes in SQL Server, the dashboard ensures real-time analytics capability.

Another important purpose of the Power BI dashboard is strategic planning. By analyzing historical leave trends, management can anticipate staffing needs, forecast workload adjustments, and design better leave policies. This analytical capability elevates the HR system from a transactional data management platform to a decision-support system.

Overall, the Power BI Workforce Analytics Dashboard enhances data-driven decision-making within the HR Management System. It bridges the gap between operational data and strategic insights, ensuring that leadership can monitor workforce patterns, optimize resources, and plan organizational growth effectively.

6.2 Data Connection Steps

1. Open Power BI Desktop.

2. Click Get Data → SQL Server.

3. Enter the server name and select HRManagementDB.

4. Load the following tables: Employees, Departments, LeaveRequests, and vw_LeaveReport (view).

5. In Model View, verify relationships: Departments → Employees → LeaveRequests.

6.3 Dashboard Visualizations:

The Power BI Workforce Analytics Dashboard in the HR Management System is designed using multiple visualizations to transform structured database records into meaningful and actionable insights. Each visualization type is carefully selected based on the nature of the data and the analytical objective it serves. The combination of KPI cards, charts, tables, and filters enables both operational monitoring and strategic workforce analysis.

The dashboard begins with KPI cards that display high-level summary metrics such as Total Employees, Total Leave Requests, and Average Leave Days. These cards provide an instant overview of the organization's workforce size and leave activity. By presenting these metrics prominently, management can quickly assess the overall HR performance without navigating through detailed reports.

Bar charts are used to represent department-wise leave distribution. This visualization helps compare leave usage across departments, making it easier to identify departments with unusually high absenteeism rates. Such comparisons support better resource allocation and operational planning. The visual contrast provided by bars allows patterns to be recognized immediately.

Line charts are incorporated to show monthly or time-based leave trends. These trend lines help in identifying seasonal patterns, peak leave periods, or recurring workforce shortages. For example, if leave requests increase consistently during specific months, management can prepare contingency staffing plans in advance. Time-series visualization is essential for predictive workforce planning.

Pie charts or donut charts are used to represent employee distribution across departments. This provides a proportional view of workforce allocation and helps understand organizational structure. By visualizing percentage-based distribution, leadership can evaluate department size balance and organizational design efficiency.

A table or matrix visualization is included to display detailed employee leave records. While charts provide summarized insights, tabular data allows HR managers to drill down into individual employee-level details. This supports audit verification, detailed analysis, and compliance documentation.

Interactive slicers are integrated to allow dynamic filtering of data based on department or date range. These filters enhance user interactivity and enable customized analysis without modifying the underlying dataset. For example, a manager can view leave trends for only one department or within a specific quarter, making the dashboard flexible and user-friendly.

Overall, the dashboard visualizations convert raw HR data into structured analytical views that support both day-to-day monitoring and long-term workforce strategy. By combining summary KPIs, comparative charts, trend analysis, and interactive filters, the Power BI dashboard transforms the HR Management System into a data-driven decision-support platform.

6.4 DAX Measures

The following DAX (Data Analysis Expressions) measures were created to power the dashboard KPIs:

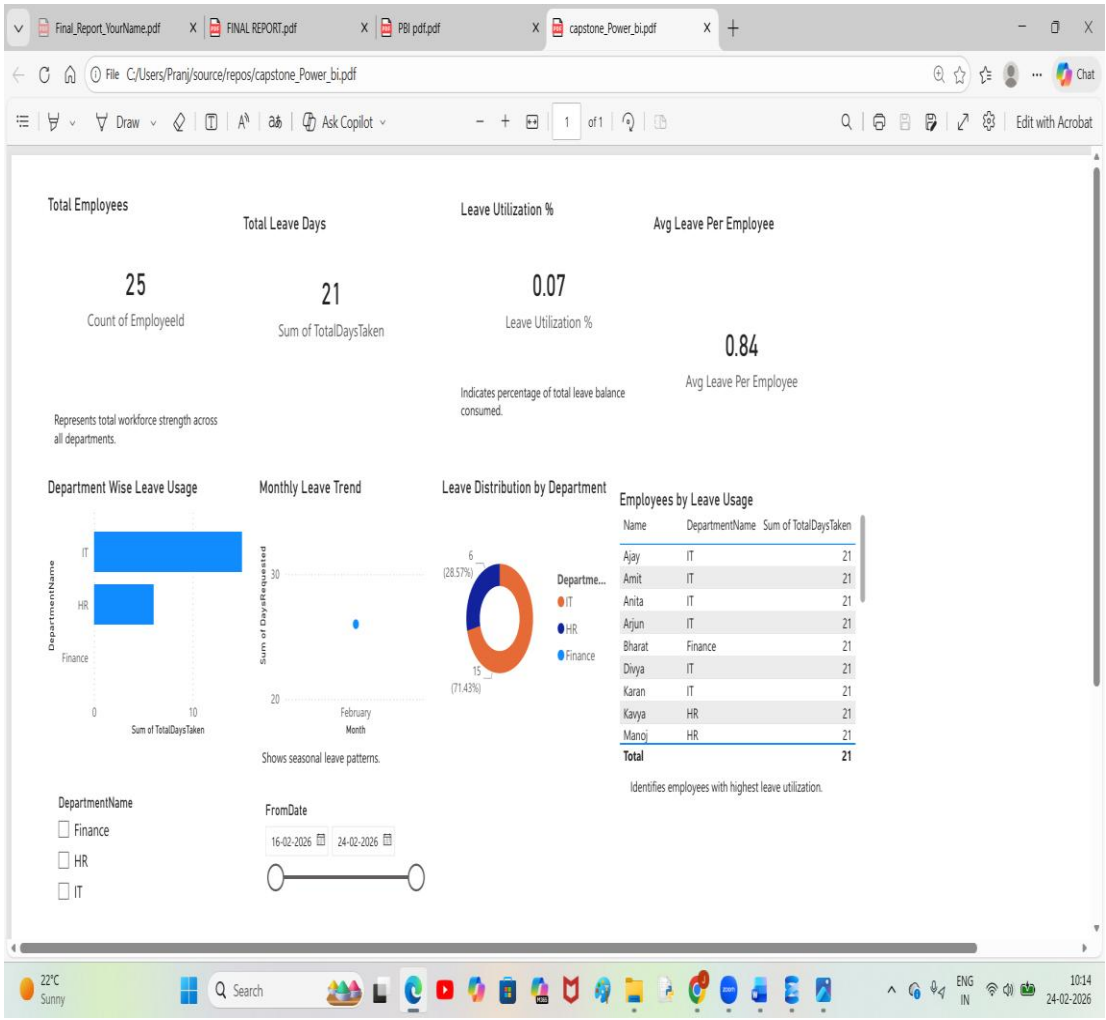
```
// Total Employees
Total Employees = COUNTROWS(Employees)

// Total Approved Leave Days
Total Leave Days =
CALCULATE(
    SUM(LeaveRequests[DaysRequested]),
    LeaveRequests[Status] = "Approved"
)

// Average Leave Per Employee
Avg Leave Per Employee =
DIVIDE([Total Leave Days], [Total Employees], 0)

// Pending Leave Count
Pending Requests =
CALCULATE(
    COUNTROWS(LeaveRequests),
    LeaveRequests[Status] = "Pending"
)
```

Dashboard Dig:



6.5 Output File

The completed dashboard is saved as HR_Workforce_Analytics.pbix and can be published to the Power BI Service for organization-wide sharing and scheduled data refresh.

---

7. Azure Cloud Architecture

7.1 Azure Architecture Overview

The HR Management System is designed to be deployed on Microsoft Azure, leveraging managed cloud services to ensure scalability, security, high availability, and reduced infrastructure management overhead.

The Azure Architecture for the HR Management & Workforce Analytics Platform is designed to demonstrate enterprise readiness while maintaining minimal and justified cloud integration. Instead of migrating the entire system to the cloud, the architecture selectively integrates Azure services where they add clear business value, particularly in identity management, scalability, automation, and optional hosting. This approach ensures cost optimization while aligning the system with modern enterprise infrastructure practices.

At the core of the Azure architecture is the concept of hybrid readiness. The primary application—comprising the ASP.NET Core Web API backend and SQL Server database—can operate in an on-premises environment. However, the architecture is designed so that these components can be deployed to Azure services when scalability or enterprise deployment is required. For example, the backend API can be hosted on Azure App Service, which provides managed hosting, automatic scaling, and high availability. This eliminates the need for manual server maintenance and infrastructure management.

For database scalability, Azure SQL Database can replace or complement the on-premises SQL Server instance. Azure SQL offers automated backups, built-in high availability, and disaster recovery capabilities. As employee data grows from hundreds to thousands of records, Azure SQL ensures performance stability and secure managed storage without complex infrastructure management.

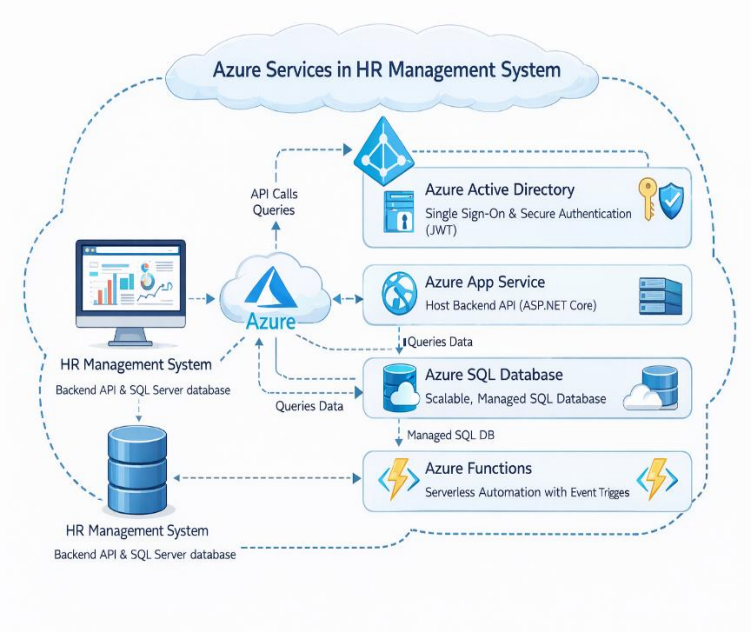
A critical component of the Azure architecture is Azure Active Directory (Azure AD). In an HR system, employee data is highly sensitive and requires secure authentication and role-based access control. Azure AD provides enterprise-grade identity management with Single Sign-On (SSO), secure token-based authentication (JWT), and centralized user management. This ensures that only authorized HR personnel, managers, or administrators can access specific functionalities of the system.

Additionally, Azure Functions are conceptually integrated for serverless automation. For example, if a leave request remains pending beyond a defined threshold (such as 72 hours), an Azure Function can automatically trigger escalation notifications. Since Azure Functions operate on a pay-per-execution model, they provide cost-efficient background processing without maintaining dedicated servers.

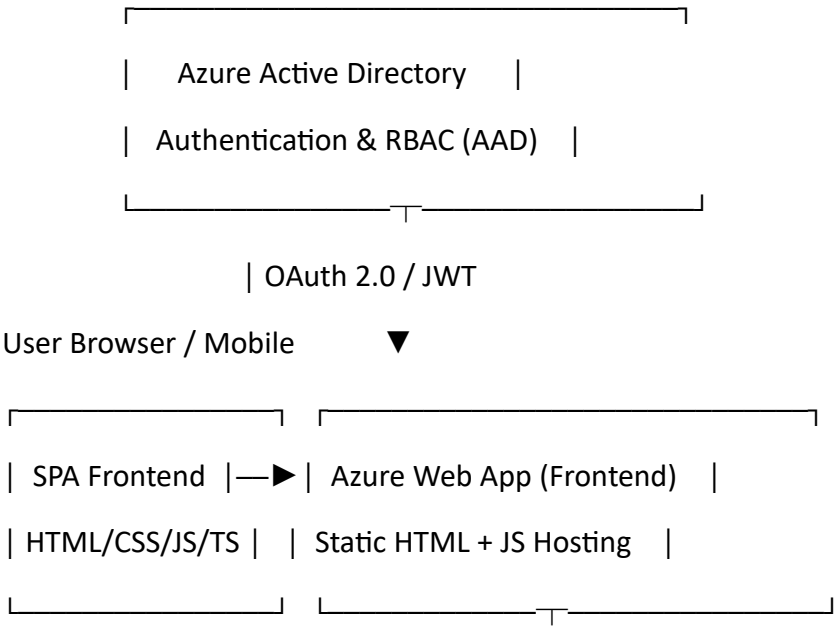
The overall Azure architecture maintains a layered flow: the frontend communicates with the backend API hosted either on-premises or in Azure App Service; the backend interacts with SQL Server or Azure SQL Database; authentication is managed through Azure AD; and automation tasks are handled by Azure Functions. This modular cloud integration ensures scalability, security, and enterprise compliance while keeping operational costs controlled.

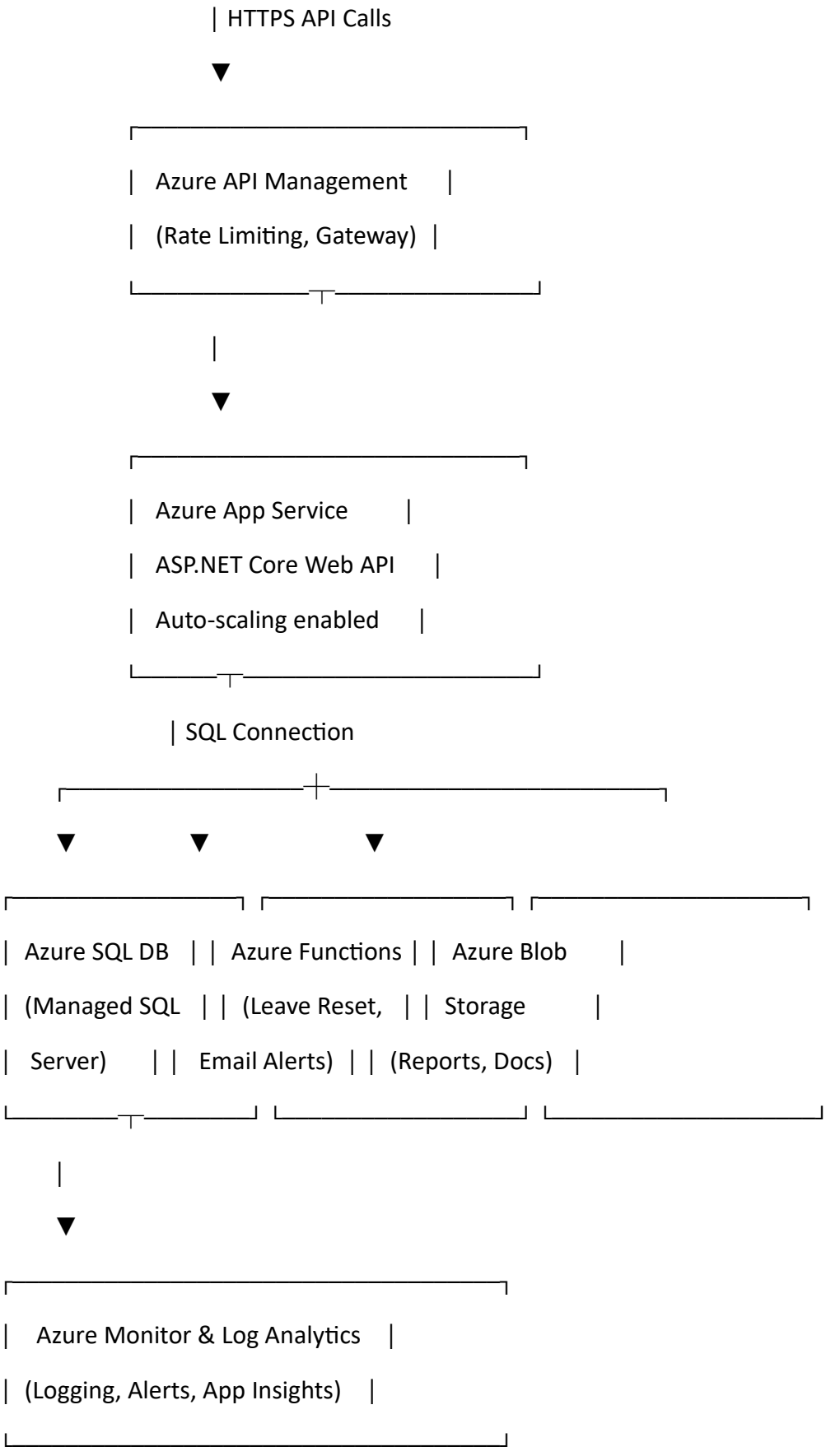
In summary, the Azure Architecture for the HR Management System supports secure identity management, scalable hosting, managed database services, and serverless automation. It ensures that the system is cloud-ready, future-proof, and capable of supporting organizational growth without requiring a complete redesign of the existing architecture.

7.2 Azure Services Used:



7.3 Azure Architecture Diagram




| HTTPS API Calls
▼
Azure API Management
(Rate Limiting, Gateway)
|
▼
Azure App Service
ASP.NET Core Web API
Auto-scaling enabled
| SQL Connection
▼ ▼ ▼
Azure SQL DB
(Managed SQL
Server)
Azure Functions
(Leave Reset,
Email Alerts)
Azure Blob
Storage
(Reports, Docs)
|
▼
Azure Monitor & Log Analytics
(Logging, Alerts, App Insights)

7.4 Security Architecture

- All data in transit is encrypted via HTTPS / TLS 1.3.

- Azure Active Directory provides centralized identity management with Multi-Factor Authentication (MFA).

- Role-Based Access Control (RBAC) limits API actions based on user roles: HR Admin, Employee, and Manager.

- Azure SQL Database uses Transparent Data Encryption (TDE) for data at rest.

- Azure API Management enforces rate limiting (100 requests per minute) to prevent abuse and DDoS attacks.

- VNet integration isolates the App Service and Database within a private Azure Virtual Network, blocking public access to the database directly.

---

8. Conclusion

8.1 Summary of Achievements

The HR Management System successfully demonstrates the end-to-end development of a full-stack enterprise application. The project covers all required modules of the capstone curriculum and applies industry-standard practices across every layer of the system. The HR Management & Workforce Analytics Platform successfully achieves the objective of transforming a traditional, manual HR process into a structured, scalable, and enterprise-ready digital solution. The project demonstrates the practical implementation of software engineering principles, database design best practices, and analytical reporting techniques within a real-world business context.

From a backend perspective, the system is developed using ASP.NET Core Web API with a layered architecture that separates models, repositories, services, and controllers. This design ensures maintainability, scalability, and adherence to SOLID principles. Business logic is centralized within the service layer, ensuring consistent validation such as duplicate

employee prevention and controlled leave management. Dependency Injection is implemented to maintain loose coupling and support future extensibility.

On the database side, the system applies normalization principles up to the Third Normal Form (3NF), ensuring data integrity and eliminating redundancy. Primary and foreign key constraints maintain referential relationships between Employees, Departments, and LeaveRequests. Indexing strategies are implemented to optimize performance for frequently queried columns, and database views are created to simplify reporting and analytics.

The frontend provides a responsive and interactive user interface that allows seamless interaction with backend APIs. By implementing dynamic form handling and SPA-like behavior, the system enhances user experience while maintaining a clean separation between presentation and business logic layers.

In terms of reporting, SSRS is integrated to generate structured, paginated operational reports suitable for administrative and compliance purposes. For advanced analytics, Power BI is used to build an interactive workforce dashboard that visualizes key performance indicators such as leave utilization trends, department-wise analysis, and monthly workforce patterns. This integration transforms the system from a transactional application into a data-driven decision-support platform.

The project also demonstrates enterprise readiness through conceptual Azure integration. Azure Active Directory is incorporated for secure identity management, Azure App Service and Azure SQL Database are considered for scalable deployment, and Azure Functions are proposed for serverless automation such as leave escalation workflows. This minimal yet justified cloud integration ensures cost efficiency while maintaining scalability and future adaptability.

Overall, the project successfully combines backend development, relational database design, frontend interaction, operational reporting, analytics visualization, and cloud readiness into a unified HR Management solution. It reflects a strong understanding of full-stack development principles and enterprise system architecture, positioning the platform as a scalable and future-ready HR solution.

8.2 Key Learnings

- Applied SOLID principles in a real-world C# project, understanding why Dependency Inversion makes systems more flexible and testable.

- Understood the importance of database normalization in preventing data anomalies at scale.

- Experienced the practical benefits of SPA architecture in reducing server load and improving user experience.

- Gained hands-on knowledge of REST API design conventions, HTTP status codes, and Swagger documentation.

- Learned to build meaningful BI dashboards using Power BI DAX measures and direct SQL Server connections.

- Developed an appreciation for cloud architecture patterns that ensure security, scalability, and high availability on Azure.

8.3 Future Enhancements

- Implement Entity Framework Core ORM to replace the in-memory repository with real database integration.

- Add JWT-based authentication and role-based authorization to secure all API endpoints.

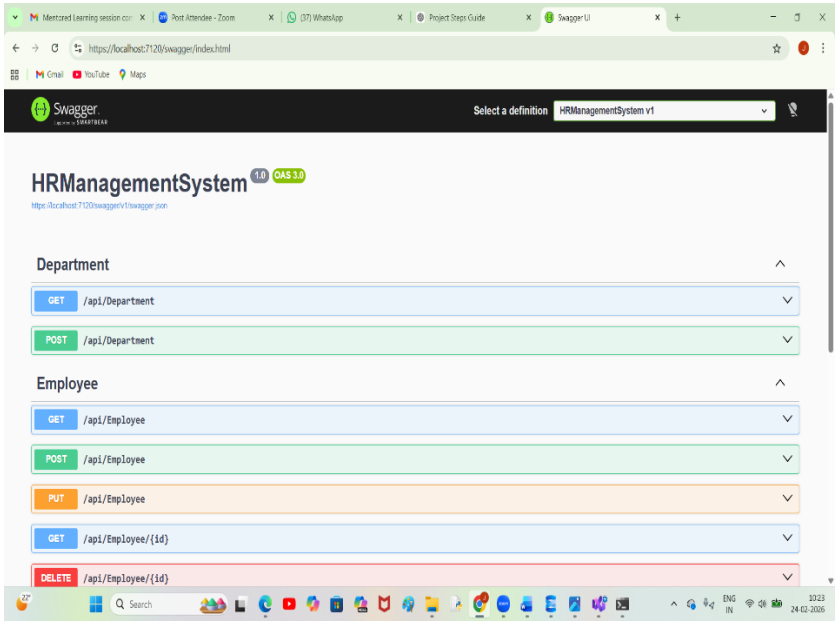
- Develop a full React or Angular frontend to replace the current vanilla JS SPA.

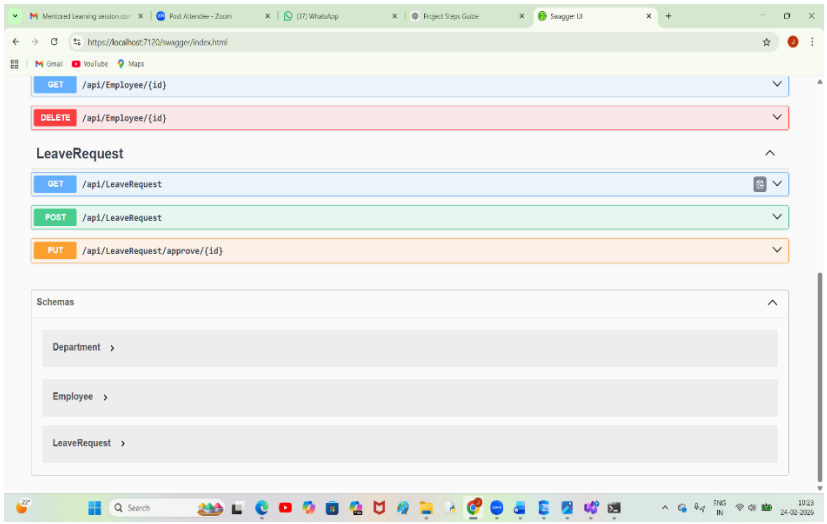
- Deploy the complete system to Azure using CI/CD pipelines via Azure DevOps.

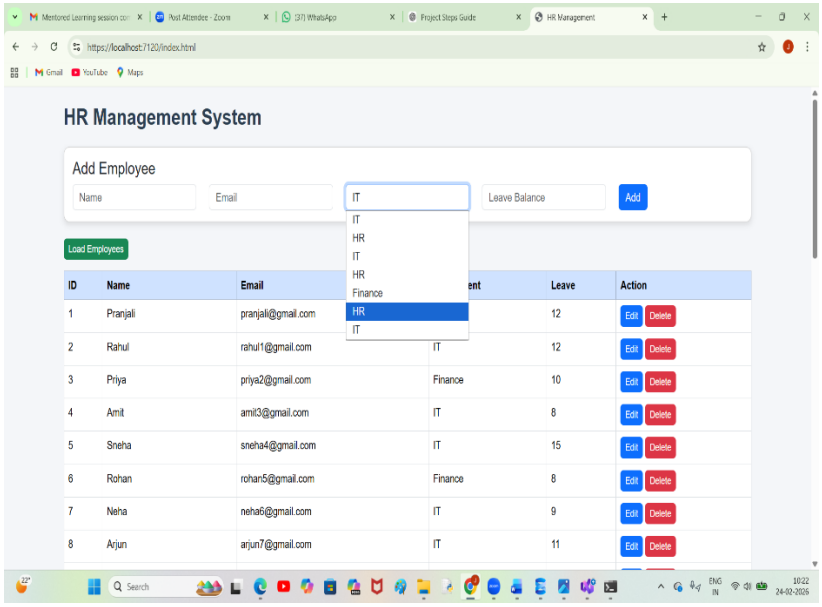
- Add email notification integration via Azure Functions when leave is approved or rejected.

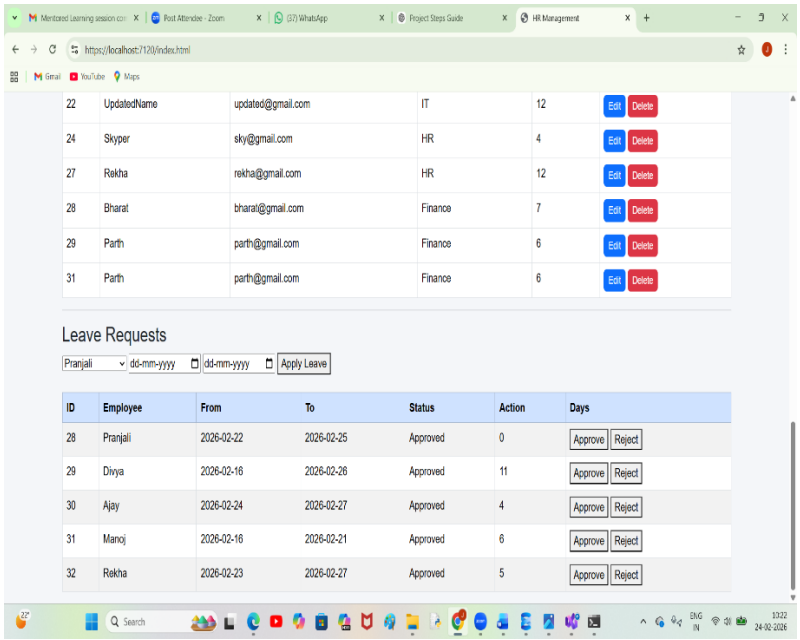
- Introduce automated unit tests using xUnit for the Service and Repository layers.

OUTPUT DIG:




https://localhost:7120/swagger/index.html
GET /api/Employee/{id}
DELETE /api/Employee/{id}
LeaveRequest
GET /api/LeaveRequest
POST /api/LeaveRequest
PUT /api/LeaveRequest/approve/{id}
Schemas
Department
Employee
LeaveRequest


https://localhost:7120/index.html
HR Management System
Add Employee
Name
Email
IT
Leave Balance
Add
Load Employees
ID Name Email Leave Action
1 Pranjali pranjali@gmail.com 12 Edit Delete
2 Rahul rahul1@gmail.com IT 12 Edit Delete
3 Priya priya2@gmail.com Finance 10 Edit Delete
4 Amit amit3@gmail.com IT 8 Edit Delete
5 Sneha sneha4@gmail.com IT 15 Edit Delete
6 Rohan rohan5@gmail.com Finance 8 Edit Delete
7 Neha neha6@gmail.com IT 9 Edit Delete
8 Arjun arjun7@gmail.com IT 11 Edit Delete


https://localhost:7120/index.html
22 UpdatedName updated@gmail.com IT 12 Edit Delete
24 Skyper sky@gmail.com HR 4 Edit Delete
27 Rekha rekha@gmail.com HR 12 Edit Delete
28 Bharat bharat@gmail.com Finance 7 Edit Delete
29 Parth parth@gmail.com Finance 6 Edit Delete
31 Parth parth@gmail.com Finance 6 Edit Delete
Leave Requests
Pranjali
dd-mm-yyyy
dd-mm-yyyy
Apply Leave
ID Employee From To Status Action Days
28 Pranjali 2026-02-22 2026-02-25 Approved 0 Approve Reject
29 Divya 2026-02-16 2026-02-26 Approved 11 Approve Reject
30 Ajay 2026-02-24 2026-02-27 Approved 4 Approve Reject
31 Manoj 2026-02-16 2026-02-21 Approved 6 Approve Reject
32 Rekha 2026-02-23 2026-02-27 Approved 5 Approve Reject